

Informe final · postopFriend

Tech Sphere Challenge 2026 · Voice Agent Edition · Source Meridian

Juan Pablo Pérez

9 de agosto de 2026

Resumen ejecutivo

postopFriend es un agente de voz que llama a pacientes recién operados, conversa con ellos en español colombiano, entiende sus síntomas apoyándose en guías clínicas reales (RAG) y decide cuándo escalar a personal humano. El LLM extrae variables clínicas; nunca decide el nivel de triage: eso lo hace un motor de reglas determinista y versionado, calibrado contra 160 casos etiquetados con 100 % de recall en los casos rojos y amarillos.

El sistema corre completo hoy: ingesta de conocimiento con alta y baja en caliente, llamada de voz en tiempo real por WebSocket con barge-in y respaldo de texto, extracción clínica con verificación de cita en código, motor de triage, máquina de estados con escalamiento por cuatro canales, y panel de observabilidad. 264 pruebas automatizadas corren en ~40 segundos sin red ni API.

Este informe declara el modelo usado y por qué, documenta las decisiones de diseño más relevantes con su alternativa descartada, reporta los resultados medidos de las tres evaluaciones automatizadas, y deja explícito qué falta y por qué. Sigue la misma regla que gobierna el resto del proyecto: ninguna cifra se publica sin el comando que la reproduce al lado.

1. El problema y el encuadre clínico

Después de una cirugía, el seguimiento telefónico depende de personal de enfermería con agenda limitada, y las señales de alarma (fiebre, dolor creciente, signos de infección de sitio quirúrgico) se pierden entre llamadas que no se hacen o se hacen tarde. El reto pide un agente que llame, converse, entienda síntomas apoyado en guías clínicas y decida cuándo escalar, sin diagnosticar ni prescribir.

postopFriend parte de ocho supuestos declarados explícitamente (detalle en el README): el sistema inicia la llamada; conoce la ficha del paciente pero no su trayectoria clínica, que solo puede averiguar conversando; no diagnostica ni prescribe, solo recoge, clasifica, informa y escala; una llamada es un caso, sin memoria entre llamadas en esta versión; los datos son sintéticos y

sin validación clínica; el navegador de referencia es Chrome o Edge; y los umbrales de triage están calibrados contra el dataset del reto, no clínicamente certificados.

La asimetría que gobierna cada decisión de diseño la declara el propio reto: un falso negativo (no escalar un caso que sí lo necesitaba) pesa más que un falso positivo (escalar de más). Esa regla aparece en el motor de triage, en el estado `INDETERMINADO` que obliga a indagar antes de cerrar, y en qué se decidió evaluar primero.

2. Declaración del modelo y cumplimiento de la compuerta G3

Modelo del agente: llama-3.3-70b-versatile, servido por Groq.

El kit oficial del reto nombra cuatro modelos permitidos y, en concreto, «Llama 3.1 70B (vía Groq)»:

Modelo permitido	Disponible en el catálogo de Groq (7 ago 2026)
Llama 3.1 70B (vía Groq)	No disponible (descontinuado)
Google Gemini 1.5 Flash	No aplica (otro proveedor)
Llama 3.2 1B/3B (vía Ollama)	No aplica (local)
Phi-3.5 Mini (vía HuggingFace)	No aplica (local)

`python scripts/check_models.py` consulta el catálogo vivo de Groq y confirma que llama-3.1-70b-versatile ya no existe entre los 15 modelos que la API devuelve. llama-3.3-70b-versatile es su sucesor directo, del mismo fabricante (Meta), servido por el mismo proveedor que el propio kit recomienda. Las tres alternativas restantes son locales o de otro proveedor, y ninguna sostiene una conversación de voz fluida en la máquina del jurado sin descargas que pondrían en riesgo la compuerta G2.

Se consultó a Source Meridian (communications@sourcemeridian.com) antes de construir sobre esta decisión, y la organización confirmó que se admite la siguiente versión disponible del modelo listado. Toda la interacción con el modelo pasa por un único archivo, `app/agent/llm.py`: cambiar de modelo o de proveedor es un cambio contenido ahí y no una reescritura del agente. Es la póliza que permitiría migrar si Source Meridian revirtiera la confirmación.

Groq Whisper large-v3-turbo transcribe la voz del paciente; edge-tts con la voz es-CO-SalomeNeural sintetiza la del agente. Ninguno de los dos es el LLM de decisión y no están sujetos a la restricción de G3.

3. Arquitectura y flujo de un turno

El diagrama completo (vista general, máquina de estados, ciclo de conocimiento y frontera del LLM) está en docs/arquitectura.md y docs/arquitectura.png, con una tabla que mapea cada caja del diagrama a su archivo real: la rúbrica advierte que el jurado toma una caja al azar y la busca en el código, y los nombres se hicieron coincidir a propósito.

Presupuesto fijo: **dos llamadas al LLM por turno**. Todo lo demás es determinista:

```
audio → Silero VAD (cliente) → STT (Groq Whisper) → router (determinista)
      → extractor clínico (LLM #1) → motor de triage (determinista)
      → máquina de estados → [si aplica] RAG híbrido → generador (LLM #2)
      → guardarraíles → [si escala] alerta por 4 canales → TTS (edge-tts) → audio
```

La decisión de diseño central: **el LLM no decide el nivel de triage**. Extrae variables clínicas a un JSON verificado; un motor de reglas versionado (app/triage/rules.yaml) calcula el nivel con precedencia bandera-roja → estado-incompleto → score. Si el LLM decidiera, la misma llamada podría dar amarillo hoy y rojo mañana sin que nadie lo note, y el sistema no se podría probar sin gastar cuota de API.

Se evaluó y descartó LangGraph para la máquina de estados: siete nodos no justifican cuarenta dependencias transitivas, un riesgo directo para la compuerta G2 de instalación en 15 minutos, ni esconder la instrumentación de latencia que el reto exige medir a mano. Es la decisión que se desarrolla en la Pregunta 2 del video.

4. RAG y conocimiento vivo · criterio de 20 pts, compuerta G5

El corpus son 107 documentos clínicos del kit oficial; 106 se indexan (uno es un escaneo sin capa de texto y se declara, no se indexa en silencio) en 9 512 fragmentos, contruidos en 7,1 minutos. La recuperación es híbrida: BM25 léxico derivado de ChromaDB en memoria más denso, fusionados por RRF, con MMR y un *boost*, nunca un filtro, por procedimiento del paciente.

Métrica (evals/run_rag_eval.py, 33 preguntas)	Resultado
hit@4	96 % (24/25)
MRR	0,733
Citas verificables	100 % (100/100)
Abstención correcta	100 % (8/8)

La decisión que más costó: la abstención **no** usa el puntaje de fusión RRF, que solo mira el puesto en la lista y da 0,650 tanto a una pregunta cubierta como a «¿quién ganó el mundial?». El coseno tampoco discrimina: una pregunta sobre mastectomía recuperaba un documento de

colecistectomía con 0,676, más alto que preguntas que el corpus sí cubre. La abstención se decide por **cobertura de término**: si una raíz específica de la pregunta (≥ 6 caracteres) tiene frecuencia documental cero en el índice, el corpus no trata el tema.

G5 (subir un documento, usarlo, borrarlo y olvidarlo) se verificó de punta a punta contra el servidor en marcha, incluida la consola en `/console` y el botón «Verificar olvido». El índice léxico se deriva de Chroma en memoria en cada cambio de `kb_version`, en vez de guardarse en disco: un archivo aparte sería un segundo sitio donde puede sobrevivir un documento borrado, que es exactamente cómo se falla esta compuerta.

Límite declarado del corpus: la carpeta `breast_cancer` del kit no contiene un solo documento sobre cáncer de mama. Sus 19 PDFs son de cáncer de cuello uterino, verificado uno por uno, así que los 8 pacientes con mastectomía se indexan sin *boost* de procedimiento, y el agente declara que no tiene la guía en vez de responder con la fuente equivocada.

5. Lógica de decisión y escalamiento · criterio de 20 pts, compuerta G4

Sobre los 160 casos etiquetados del dataset, alimentando el motor con la trayectoria clínica exacta, sin pasar por el extractor, para fijar el techo del sistema:

esperado obtenido	verde	amarillo	rojo
verde (n=123)	111	12	0
amarillo (n=25)	0	25	0
rojo (n=12)	0	0	12

Exactitud 92,5 % · **recall de rojo 100 %** · **recall de amarillo 100 %** · **falsos negativos 0**. Los 12 verdes sobre-escalados a amarillo son el precio consciente de la asimetría del reto: las dos clases se solapan en el rango de score 2-3 del propio dataset, y no existe umbral que las separe sin perder amarillos.

El estado `INDETERMINADO` es la respuesta al sub-criterio de ambigüedad: no es un nivel clínico, es la confesión de que faltan datos (dolor, fiebre o herida), y la máquina de estados no puede cerrar la llamada mientras falten. Se indaga antes de decidir, y tras dos reintentos se escala por precaución.

El sistema completo, que es la cifra que importa. La tabla anterior fija el techo: mide las reglas con los valores exactos, no lo que el paciente entrega. Pasando el extractor real sobre los diálogos (`evals/run_triage_eval.py`), sobre una muestra sesgada a propósito hacia lo grave —

rojo 24/24, amarillo 45/50, verde 40/246, n=109, la cuota no dio para más y los denominadores se citan siempre:

esperado obtenido	verde	amarillo	rojo
verde (n=40)	27	13	0
amarillo (n=45)	4	41	0
rojo (n=24)	1	9	14

Recall de rojo **58,3 %**, de amarillo 91,1 %. La exactitud global de esta muestra no es comparable con el 92,5 % de arriba, porque la mezcla de etiquetas no es la del dataset.

Ese 58,3 % es el número más incómodo del informe y por eso se desglosa en vez de suavizarlo. De los 24 rojos: 14 clasificados rojo, **9 escalados a amarillo** y 1 cerrado en verde. Los 9 se revisaron uno por uno: son pacientes de arquetipo *evasivo* y **en ninguno de los 9 el paciente llegó a decir la cifra** que el dataset da por real — «uy, no sé, no le he puesto mucho cuidado a eso... aunque sí me he sentido como acalorada a ratos», con fiebre real de 38,0. El extractor devuelve null en lugar de inventarla, y el motor, con variables críticas pendientes y reintentos agotados, escala por precaución en vez de cerrar en verde. **23 de los 24 casos rojos terminan escalados**. La métrica cuenta 10 fallos; el sistema dejó pasar uno.

Ese uno es caso_tray_pac_42_00017_7, arquetipo *minimizador_sintomas*: dolor real 9 declarado como «un poquito molesto no más, nada del otro mundo, uno aguanta», y fiebre real 37,9 declarada como «marcó como 37 y algo». El extractor leyó 3 y 37,0 — defendible sobre lo dicho, salvo que **«37 y algo» → 37,0 redondea hacia el lado inseguro**. La corrección (resolver numéricos ambiguos hacia arriba, o bajar la confianza para que el motor los trate como pendientes) queda en §11 y no se aplicó antes del cierre por una razón explícita: el caché de la evaluación se indexa por el hash del prompt del extractor, así que tocarlo invalida las 109 mediciones y no había cuota para rehacerlas.

La conclusión que se defiende es esta: la brecha entre el 100 % del motor y el 58,3 % del sistema **no mide un defecto de las reglas, mide que un paciente real no siempre dice lo que le pasa**. Un clasificador se evalúa contra la verdad; una conversación, contra lo que el interlocutor quiso contar. Diseñar para la segunda es lo que obliga a que «faltan datos» sea un estado del sistema y no un valor por defecto.

Se comprobó que el modelo no es la causa. Se repitió la evaluación completa con *gemini-3.5-flash-lite* (familia Gemini Flash, también permitida por G3) sobre los mismos 24 casos rojos: recall de rojo 62,5 % frente a 58,3 %, un caso de diferencia, y con el doble de sobre-escalamiento en verde (52,6 % frente a 32,5 %). Los mismos casos se pierden por la misma razón. El límite lo pone lo que el paciente dice, no quién lo lee.

Un bug corregido el mismo día de este informe, y por qué queda documentado. Cruzando la base de datos contra los logs se descubrió que 3 llamadas reales terminaron en nivel rojo y 14 turnos llegaron a estado Emergencia, pero la tabla alertas tenía cero filas: la emergencia se detectaba y no se avisaba a nadie. La causa era un guardián en `transicion()` (`app/agent/flow.py`) que se disparaba de nuevo en cada turno posterior a la emergencia. El estado clínico es acumulativo, una bandera roja no se cae sola, y el `return` se adelantaba para siempre al `match` que lleva a Escalar. La corrección añade una marca `emergencia_declarada` que corta el protocolo una sola vez, y dos pruebas nuevas cubren la ruta real. Queda como el ejemplo central de una regla que gobierna todo el proyecto: **una prueba que pasa sobre un caso sintético no prueba que la ruta real se recorra**; esto se detectó cruzando datos de llamadas reales contra la base, no en una suite unitaria.

Cuando escala, Alerta se persiste por cuatro canales (SQLite, JSON, Markdown y webhook) y el acta de cierre de 10 secciones queda disponible en el panel y para descarga.

6. Calidad de la conversación de voz · criterio de 15 pts, compuerta G4

La llamada corre por WebSocket con Silero VAD en el cliente (detección de fin de habla, sin CDN, empaquetado en el repo), barge-in, respaldo de «pulsar para hablar» si el VAD falla, y un campo de texto que recorre el mismo *pipeline* completo sin micrófono.

Ejercitada con **12 llamadas reales, 89 turnos con audio, 156,5 segundos de voz**. La cadena STT→TTS se verificó de ida y vuelta con **100 % de coincidencia de palabras** (`scripts/probar_voz.py`), incluida «apendicectomía».

Los guiones de emergencia, inyección detectada y «no disponible» no pasan por el LLM: salen del caché de audio pregrabado en milisegundos y no dependen de que haya cuota de API en ese instante. Es una decisión de disponibilidad, no solo de latencia.

7. Guardarraíles

Las tres penalizaciones que la rúbrica nombra explícitamente, evaluadas caso por caso sobre 31 escenarios (`evals/run_safety_eval.py`):

	Resultado
Inyección de prompt (voz y fuentes)	10/10 atrapadas
Dosis o fármacos sin respaldo	3/3

	Resultado
Tranquilizar ante bandera roja	3/3
Salirse de la misión	3/3
Turnos legítimos respetados	12/12
Falsos negativos · falsos positivos	0 · 0

Doce de los 31 casos son legítimos y tienen que pasar: un filtro que bloquea todo saca 100 % en ataques y es inservible. Esta misma evaluación encontró que el patrón de inyección bloqueaba frases normales como «Eres muy amable, gracias». Se corrigió exigiendo que la frase nombre el rol que intenta asignar, que es lo que distingue una inyección de habla cotidiana.

8. Métricas de latencia, consumo y costo

Latencia, medida sobre 49 turnos con el reloj del navegador, de punta a punta: desde que el VAD detecta que el paciente calló hasta el evento `playing` del audio.

P50	4 455 ms
mínimo	815 ms
máximo	5 809 ms
objetivo declarado	1 500 ms

Advertencia de integridad, declarada a propósito: la mayoría de estos turnos se midieron con la ruta alterna por OpenRouter (§10), que añade un salto de red frente a Groq directo. El salto explica parte de la diferencia contra el objetivo, no toda; hay trabajo real de latencia pendiente, y el desglose por etapa (`etapas_json`) es donde se va a mirar primero. Antes del cierre se remide con `LLM_BACKEND=groq` y se publica con `scripts/report_metrics.py`, que nunca escribe una cifra a mano.

Costo y consumo: cada extracción clínica cuesta ~2 800 tokens; con Groq a precio de OpenRouter (\$0,59 / \$0,79 por millón de tokens de entrada/salida), la evaluación completa de 320 llamadas sale por ~USD 0,55. Presupuesto de invocaciones: 2 llamadas al LLM por turno, 0 llamadas para los guiones fijos.

9. Cómo trabajé con IA

Usé Claude Code como asistente de desarrollo durante los tres días de construcción, con dos reglas fijas: **el LLM del producto nunca decide el triage** (eso lo revisé y lo probé yo, no se

delegó al asistente) y **ninguna cifra entra al README o a este informe sin el comando que la reproduce al lado.**

El prompt del extractor clínico (`app/agent/prompts/extractor.md`) pasó por tres iteraciones versionadas en el propio repo:

- **v1** no exigía evidencia citada, y el modelo rellenaba variables con lo esperable para ese día postoperatorio en vez de dejarlas vacías.
- **v2** exigió cita literal; el modelo empezó a parafrasear la cita para que encajara, así que se añadió verificación en código (`extractor.evidencia_verificada()`) que descarta el valor si la cita no aparece de verdad en la transcripción. El prompt pide, el código verifica.
- **v3** corrigió que pacientes que minimizan («estoy bien», «normal») se leyeran como valores reales en vez de `null`, lo que cerraba llamadas incompletas en verde.

Cada iteración salió de un fallo medido por las evaluaciones, no de una intuición: la v1 se detectó en la primera corrida de `run_triage_eval.py`, que dio 20 % de exactitud con todas las variables en `None`. Resultó ser un límite de cuota de la API mal cacheado como resultado válido, y la lección quedó como regla del proyecto (ver `docs/estado-del-proyecto.md` §7).

Lo que revisé a mano en cada sesión: cada decisión de arquitectura documentada en las secciones 3 a 5 de este informe, cada cifra antes de publicarla, y el diagrama contra el código línea por línea antes de darlo por cerrado. Lo que salió mal y quedó documentado como lección, no oculto: varios bugs de producción encontrados por evaluación o por cruce de datos, no por revisión de código (detalle completo en `docs/estado-del-proyecto.md` §7).

El historial de commits, 31 commits atómicos en español repartidos entre el 7 y el 9 de agosto, cada uno explicando el porqué, es evidencia adicional de proceso y se revisa con `git log`.

10. Entorno, restricciones de cuota y ruta alterna

Groq nivel gratuito impone 1 000 peticiones por día, 12 000 tokens por minuto y, un dato que no aparece en ninguna cabecera y solo se ve en el mensaje de un error 429, **100 000 tokens por día**, que se reponen gota a gota (~1,5 extracciones por hora) y no de golpe a medianoche. La evaluación completa de 320 llamadas necesitaría nueve días de esa cuota.

La salida: `LLM_BACKEND=openrouter`, que sirve el mismo modelo (`meta-llama/llama-3.3-70b-instruct`) por el mismo proveedor de origen (Groq), facturado por un intermediario que sí acepta tarjeta, fijado con `provider: {order: ["groq"], allow_fallbacks: false}` y verificado en cada respuesta: si OpenRouter enrutara a otro backend, la respuesta se descarta en vez de

medirse. La ruta de producción de la llamada en vivo no cambia: sigue siendo groq por defecto, y el flujo de voz en tiempo real ni siquiera ofrece la alterna.

11. Limitaciones conocidas y trabajo siguiente

- **Evaluación del sistema completo** (extractor real + motor) sobre los 320 casos: en curso, limitada por la cuota gratuita de Groq (§10); a la fecha de este informe corre por lotes con caché incremental (--solo-cache). Medidos 109 de 320, con los 24 rojos completos: la cobertura de rojo es del 100 % y la de verde del 16 %, que es el orden correcto de prioridad si la cuota no alcanza.
- **Numéricos ambiguos redondeados hacia el lado inseguro.** El extractor resuelve «marcó como 37 y algo» a 37.0. Es el único fallo real de los 24 casos rojos (§5) y la corrección es acotada: instruir al prompt que ante un valor vital dudoso resuelva hacia arriba, o baje la confianza para que el motor lo trate como pendiente y siga indagando. No se aplicó antes del cierre porque el caché de run_triage_eval.py se indexa por el hash del prompt del extractor: cambiarlo invalida las 109 mediciones y la cuota restante no daba para rehacerlas. Cambiarlo a ciegas, sin poder remedir, habría sido peor que documentarlo.
- **Latencia por la ruta de producción** (groq, sin el salto de OpenRouter): pendiente de remedir antes del cierre; ver la advertencia de §8.
- **Memoria entre llamadas:** cada llamada es un caso independiente en esta versión; el histórico se persiste y se muestra en el panel, pero no condiciona la conversación siguiente del mismo paciente.
- **Moduladores de riesgo por comorbilidad** (diabetes, obesidad) están implementados en rules.yaml pero apagados: las etiquetas del dataset no reflejan esa comorbilidad, y encenderlos duplica las falsas alarmas sin ganar sensibilidad.
- **Datos sintéticos, sin validación clínica,** y umbrales calibrados contra un dataset de referencia, no certificados clínicamente. Aviso visible en la interfaz, el README y este informe.
- Con dos semanas más: cerrar la evaluación completa de 320 casos, remedir latencia por la ruta de producción, y explorar memoria acotada entre llamadas del mismo paciente sin romper el aislamiento entre casos.

12. Reproducibilidad

Todo lo reportado aquí se reproduce con un comando:

```
python scripts/doctor.py           # diagnóstico completo
python scripts/check_models.py      # evidencia de G3 contra la API viva
python -m pytest                    # 264 pruebas, ~40 s, sin red
python evals/run_engine_eval.py --fallos --guardar
python evals/run_rag_eval.py
python evals/run_safety_eval.py
python scripts/probar_voz.py        # voz de ida y vuelta sin micrófono
python scripts/report_metrics.py    # latencia, tokens y costo del README
```

Diagrama de arquitectura: docs/arquitectura.png (fuente y tabla caja→archivo en docs/arquitectura.md). Video demo: *(pendiente, ver README)*. Capturas del demo: docs/evidencia/ *(pendientes de captura antes del cierre)*.

Documento de traspaso técnico completo, con cada decisión, cifra y bug en su contexto: docs/estado-del-proyecto.md.